Representation Learning for Rankings

Lecture 6: Learning to Rank — From Pointwise to Listwise

Avishek Anand

Information Retrieval

March 2, 2026

Outline

1. Recap & Motivation — Why Learn a Ranker?
2. Evaluation Metrics — What Are We Optimising?
3. Pointwise Approaches
4. Pairwise Approaches
5. LambdaRank & LambdaMART
6. Listwise Approaches

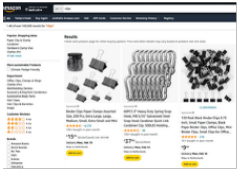
Section 1

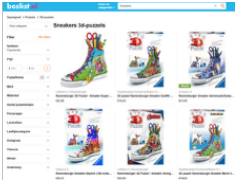
Recap & Motivation — Why Learn a Ranker?

Relevance is Multi-faceted

Industrial search engines must optimize for more than just "topical" relevance.

- **Textual**: BM25, semantic match.
- **Quality**: Ratings, reviews.
- **Business**: Price, Prime shipping, revenue per click.





The Scale of Industrial Ranking

Modern search giants handle a massive explosion of signals.

- **Scope**: Amazon, eBay, and Google use **1000+ features** per document.
- **The Wall**: No human can manually tune 1000 weights $\mathbf{w}$ to optimize a metric like NDCG.
- **Heterogeneity**: Combining BM25 (float) with "Is Fresh" (binary) and "Price" (currency).

The LTR Necessity

We need a principled, automated way to learn the optimal "recipe" for combining these signals.

A Document as a Feature Vector

Learning to Rank represents each query-document pair as a vector x.

$$\Phi(q, d) = \underbrace{\begin{array}{ccccc} x_1 & x_2 & x_3 & \cdots & x_n \\ \text{BM25 Score} & \text{PageRank} & \text{Click Count} & & \text{Freshness} \end{array}}_{\mathbf{x}_{q,d} \in \mathbb{R}^{1000+}}$$

The LTR Goal

Learn $f(\mathbf{x}) \rightarrow$ score for optimal ranking

Where LTR Fits: The Multi-Stage Pipeline

LTR is usually the final "re-ranking" stage in a complete stack.

Stage 1: Sparse Retrieval (L3) *or* Dense Retrieval (L5)

↓

Optional: Query Expansion / PRF improves recall (L3)

↓

Stage 2: Neural Re-ranking / Cross-Encoder (Today)

↓

Stage 3: Learning to Rank (LTR) combines all signals (L6)

↓

Final Ranked List → User or RAG Application

Recap: Why LTR is Necessary

Our journey moved from hand-crafted matchers to complex neural-classical ensembles.

- **L3–L4**: BM25 and BERT Cross-Encoders (Standard 2-stage pipeline).
- **L5**: Dense retrieval with Bi-Encoders and Late Interaction (ColBERT).
- **L5 (cont)**: Generative retrieval (DSI, SEAL).

The Unasked Question

How do we **train** these components together to optimize for the final ranking metrics we actually care about (NDCG, MAP)?

Today's Goal: Directly Optimizing Rankings

The core problem is learning a scoring function that respects document ordering.

1. **Pointwise**: Treat relevance as independent regression/classification.
2. **Pairwise**: Learn relative preferences between pairs of documents.
3. **Listwise**: Optimize the entire ranked permutation at once.

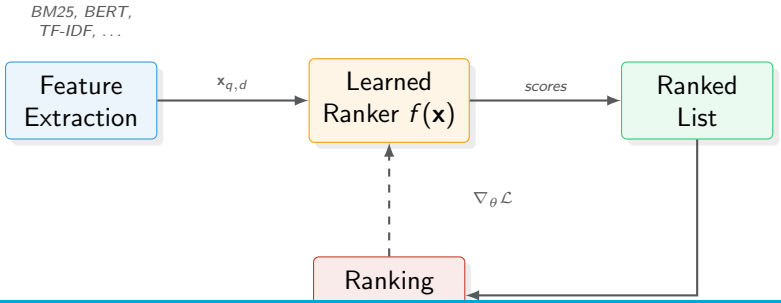
The Non-Differentiable Barrier

Ranking metrics depend on **sorting**, which is discrete. We must find differentiable surrogates or smooth approximations to use Gradient Descent.

The Learning to Rank Pipeline

LTR separates feature extraction from the learned scoring and optimization logic.

- **Traditional**: Hand-crafted features (BM25, PageRank) + LambdaMART.
- **Neural**: End-to-end representation learning + Ranking loss.
- **Cycle**: Feedback loop via gradients from ranking loss → model parameters.



Section 2

Evaluation Metrics — What Are We Optimising?

Recap: Binary Metrics

Traditional metrics focus on whether a document is "relevant" or not.

- **P@k / R@k**: Good for broad recall but position-blind within top-k.
- **MRR**: Excellent for navigational/QA tasks (cares about first result).
- **MAP**: Averages precision across all relevant docs; sensitive to rank.

The Limitation

Binary metrics ignore the **intensity** of relevance. In modern search, we often have graded labels (e.g., Perfect, Good, Fair, Bad).

NDCG: The Gold Standard for LTR

NDCG handles graded relevance and prioritizes top-heavy rankings.

- **Grades**: Cumulative Gain (CG) sums relevance scores: $\sum \text{rel}_i$.
- **Discount**: DCG applies a logarithmic decay: $\sum \frac{\text{rel}_i}{\log_2(i+1)}$.
- **Normalization**: NDCG scales DCG to $[0, 1]$ relative to the ideal sort.

$$\text{NDCG@}k = \frac{\sum_{i=1}^{k} \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)}}{\text{IDCG@}k}$$

Worked Example: Computing NDCG@5

Let's calculate the cost of a "miss" in the top positions.

- **Current**: Grades $[2, 3, 1, 0, 0] \rightarrow \text{DCG} \approx 4.39$.
- **Ideal**: Grades $[3, 2, 1, 0, 0] \rightarrow \text{DCG} \approx 4.76$.
- **Result**: $\text{NDCG} = 4.39/4.76 \approx \mathbf{0.92}$.

The Logarithmic Effect

Notice that moving the "3" from position 1 to 2 caused an 8% drop in score. NDCG is extremely sensitive to errors at the very top.

Section 3

Pointwise Approaches

Pointwise: Ranking as Classification/Regression

The simplest approach treats each document-query pair as an isolated task.

- **Regression**: Predict the exact relevance grade y (e.g., MSE loss).
- **Classification**: Predict relevance class (e.g., Binary Cross-Entropy).
- **Scalability**: $O(N)$ complexity makes it fast for massive candidate sets.

The Fatal Flaw

Pointwise models ignore the **relative order** between documents. They don't know that ranking $d_1 > d_2$ is what actually matters, not the exact score values.

Pointwise in Disguise: Standard Cross-Encoders

Recall monoBERT from Lecture 4 is technically a pointwise ranker.

- **Input**: Joint query-document sequence `[CLS] Q [SEP] D`.
- **Loss**: Binary Cross-Entropy on the `[CLS]` representation.
- **Success**: Works well because BERT captures deep semantics, but **not** because the loss is optimal for ranking.

The Capacity Paradox

With enough data and model capacity (like LLMs), simplistic pointwise losses often reach surprisingly high performance.

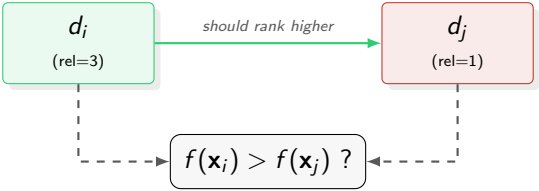
Section 4

Pairwise Approaches

Pairwise: Relative Preferences

Instead of absolute scores, we learn which document is "better" in a pair.

- **Intuition**: For each pair (d_i, d_j), if $y_i > y_j$, the model should score $s_i > s_j$.
- **Strength**: Transforms ranking into binary classification of pairs.
- **Feedback**: Errors are symmetric — push one up, the other down.



Goal: Maximize agreement with labels

RankNet: Learning from Pairs

RankNet models the preference probability between two documents.

- **Probability**: $P(d_i \succ d_j) = \sigma(s_i - s_j)$.
- **Loss**: Binary Cross-Entropy on pairs where d_i is more relevant.
- **Update**: Gradient pushes s_i up and s_j down proportional to error.

$$\mathcal{L}_{ij} = \log\left(1 + e^{-(s_i - s_j)}\right)$$

Worked Example: RankNet Misstep

Suppose a Blog (rel=1) is scored at 2.5, while a Tier-1 list (rel=3) is at 2.1.

- **Conflict**: $P(\text{Tier-1} \succ \text{Blog}) \approx 0.40$ (Model is wrong).
- **Gradient**: Pushes Tier-1 score up ($+0.60$) and Blog down (-0.60).
- **Observation**: RankNet weights all swaps equally, regardless of position.

The Position Blindness

Swapping docs at positions 1–2 should be "more important" than positions 99–100. RankNet's loss doesn't know about positions.

RankSVM: Margin-Based Ranking

Historical significance: First to use implicit feedback (clicks) for LTR.

- **Margin**: Force $\mathbf{w}^\top(\mathbf{x}_i - \mathbf{x}_j) \geq 1$ for correctly ordered pairs.
- **Feed**: Users clicking result i after skipping $j \implies d_i \succ d_j$.
- **Legacy**: Solid theory (max-margin) but less scalable than boosting.

Key Contribution

Joachims (2002) showed that even noisy implicit signals (clicks) can train a competitive ranker when framed as a pairwise problem.

Section 5

LambdaRank & LambdaMART

LambdaRank: Metric-Aware Gradients

The breakthrough: Weighting pair updates by their impact on NDCG.

- **Weight**: Multiply the pairwise gradient by $|\Delta\text{NDCG}|$ of the swap.
- **Focus**: Errors at the top (pos 1–2) get massive gradients.
- **Result**: Errors at the bottom (pos 99–100) are almost ignored.

$$\lambda_{ij} = -\frac{1}{1 + e^{s_i - s_j}} \cdot |\Delta\text{NDCG}(i, j)|$$

The Lambda Insight

LambdaRank defines gradients directly, without an explicit loss function.

- **Problem**: NDCG is non-differentiable (jumpy).
- **Solution**: Specify the physical "force" (λ) acting on each doc.
- **Analogy**: "Pull" relevant docs up, "push" irrelevant ones down.

A Theory Mystery

Surprisingly, optimizing these gradients leads to a local optimum for the target ranking metric (Burges et al., 2006).

Worked Example: Lambda Power

Correcting errors at the top yields much higher "gravitational" force.

- **Top Swap** (Pos 1–2): Resulting $|\Delta \text{NDCG}| = \mathbf{0.154}$.
- **Bottom Swap** (Pos 3–4): Resulting $|\Delta \text{NDCG}| = \mathbf{0.030}$.
- **Impact**: The top swap exerts $5\times$ more gradient force on the model than the bottom swap.

Convergence

This weighting causes the model to "give up" on tail documents and focus exclusively on perfecting the top-10 results.

LambdaMART: Industry Workhorse

Combines Lambda gradients with Gradient Boosted Regression Trees.

- **MART**: Multiple Additive Regression Trees (Boosting).
- **Process**: Compute current λ for all docs $\rightarrow$ Fit a tree to predict those λ values $\rightarrow$ Add tree to ensemble.
- **Standard**: Powering Bing, Yahoo, and specialized e-commerce search for years.

Why Trees?

Non-neural trees handle **heterogeneous features** (BM25 score + PageRank + Category ID) much better than MLPs without massive feature engineering.

LTR Features: What the Model "Sees"

Traditional LTR models use a large bank of query-document features.

- **Lexical**: BM25, TF-IDF, Language Model scores.
- **Structural**: In-link count (PageRank), URL length, title matches.
- **Modern**: Dense similarity (L5), Cross-encoder scores (L4).

The Ensemble Effect

LambdaMART doesn't replace BM25; it uses BM25 as **one of many inputs**. It learns the optimal "mixture" of classical and semantic signals.

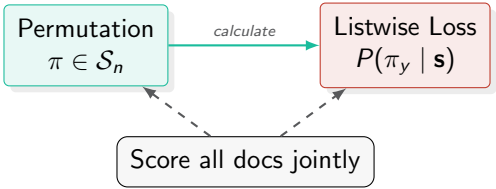
Section 6

Listwise Approaches

Listwise: Optimising the Distribution

Listwise models treat the entire set of documents as a single unit.

- **Philosophy**: Don't look at doc pairs; look at the *permutation*.
- **Models**: $P(\text{ranking} \mid \text{scores})$ is modelled as a probability distribution.
- **Loss**: Minimize KL-divergence between predicted and ideal distributions.



Goal: Minimize distance between predicted and ideal permutations

ListNet: Top-One Probabilities

ListNet models the probability that document i is ranked at position 1.

- **Probabilistic**: $P_s(d_i) = \frac{\exp(s_i)}{\sum_j \exp(s_j)}$ (Standard Softmax).
- **Loss**: Cross-Entropy between predicted and target softmaxes.
- **Benefit**: Naturally handles the interaction between all documents.

Intuition

Pushing one document to position 1 automatically lowers the "Top-1" probability of all other documents in the candidate set.

Connection: Knowledge Distillation as LTR

Teaching a Bi-Encoder (Student) to mimic a Cross-Encoder (Teacher) is a ranking task.

- **Teacher**: Provides graded relevance scores for top-k documents.
- **Student**: Must learn to reproduce the relative ordering, not just absolute scores.
- **Losses**: Listwise KL-divergence over softmaxes works best.

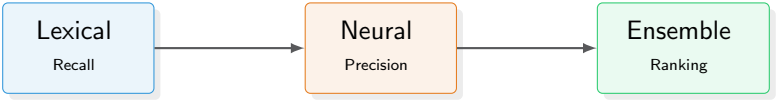
TAS-B Result

Using pairwise/listwise distillation significantly outperforms simple MSE on the MS MARCO benchmark.

The Modern Retrieve-Then-Rerank Stack

Modern search engines are multi-stage ensembles of different LTR paradigms.

- **Stage 1**: Dense/Sparse Retrieval (Recall optimized, Pointwise InfoNCE).
- **Stage 2**: Neural Re-ranker (High interaction, Pairwise BERT).
- **Stage 3**: LambdaMART (Aggregates all signals + metadata).



Practical LTR: LightGBM & PyTerrier

Standardized libraries make it easy to train metric-aware rankers.

- **LightGBM**: The primary engine for LambdaMART in industrial search.
- **PyTerrier**: Orchestrates the pipeline from indexing to ensemble training.
- **Objective**: Simply set `"objective": "lambdarank"` to start.

Feature BANK

The power of LambdaMART comes from its ability to use **50+ features**: BM25, PageRank, BERT scores, and URL statistics.

Summary: Choosing Your Paradigm

The trade-off is between ranking awareness and computational complexity.

- **Pointwise**: Fast, scalable, but position-blind. Best for 1st stage.
- **Pairwise**: The standard for re-ranking. LambdaMART is the workhorse.
- **Listwise**: Use for deep semantic interaction or small top-k re-ranking.

The Recall Ceiling

Remember: A perfect re-ranker cannot fix what the first stage missed. LTR is most effective when the retrieval stage has high recall.

Next Class: Evaluation & Large Scale Systems

Next week, we move into the "wild" — how to evaluate systems with real users.

- **Offline**: Benchmarks like TREC and the limits of static labels.
- **Online**: A/B testing, Interleaving, and Click-through analysis.
- **Deployment**: Building low-latency production pipelines.

Preparation

Review the PyTerrier documentation for the Learning to Rank module.

Backup: Smooth Ranks (ApproxNDCG)

ApproxNDCG makes sorting differentiable by using sigmoids.

- **Problem**: Hard ranks are discrete (zero gradient).
- **Sigmoid**: Replace $\mathbb{1}[s_j > s_i]$ with $\sigma(\alpha(s_j - s_i))$.
- **Result**: A differentiable surrogate for the NDCG metric.

$$\hat{\text{rank}}(d_i) = 1 + \sum_{j \neq i} \sigma\big(\alpha \cdot (s_j - s_i)\big)$$